In the name of God

UNIVERSITY OF TEHRAN

Artificial Intelligence

Parham Javan

ID: 810800008

CA 2

Problem 2

Connect4 Game and Minimax Algorithm

Connect4 Game Description:

Connect4 is a two-player strategic game where players take turns dropping pieces into a 6x7 grid. The goal is to be the first to connect four pieces in a row, column, or diagonal. If the grid fills without a four-in-a-row connection, the game results in a tie.

Minimax Algorithm with Alpha-Beta Pruning:

In this project, the minimax algorithm was implemented with depth limitation and optional alpha-beta pruning to determine the optimal moves for the CPU player. The CPU is programmed to maximize its score while minimizing the opponent's (player's) score based on the board's evaluation function. A heuristic function evaluates the board state at non-terminal nodes, considering factors like potential connect-fours, three-in-a-rows, and two-in-a-rows to estimate each move's effectiveness.

Report on Results

1. Depth Effect on Performance and Winning Probability:

The minimax depth significantly impacts performance, win rate, and computation time. As the depth increases:

- Winning Probability: The CPU's winning probability typically increases since deeper searches allow for more foresight in move selection.

- Execution Time: Computation time increases exponentially with each depth increase due to the higher number of possible game states examined.

- Explored Nodes: With deeper searches, the algorithm explores a greater number of nodes, especially without pruning.

2. Effective Ordering for Alpha-Beta Pruning:

Ordering child nodes can indeed improve alpha-beta pruning efficiency. Prioritizing moves with the highest heuristic scores first (like center moves or moves that block an opponent's winning chance) increases the chances of pruning. This method is not perfect but can significantly reduce the branching factor by allowing earlier cuts.

3. Branching Factor Behavior in Connect4:

The branching factor is the average number of possible moves per position. In Connect4, this starts high but decreases as columns fill up. Early in the game, the branching factor is around 7 (all columns are open), but it gradually drops as fewer columns are available, reducing the search space.

4. Speed Gains with Alpha-Beta Pruning:

Alpha-beta pruning accelerates the minimax algorithm by ignoring branches that don't affect the final decision, allowing the algorithm to skip unnecessary nodes without compromising accuracy. This process reduces the number of evaluated nodes, particularly in deeper search depths, by ensuring that only the most relevant game states are examined.

And as a result, we observe disabled pruning has significantly more taken time

Check result

```
Depth: 1 | Pruning: Enabled -> User Wins:  13, CPU Wins:  7, Ties:  0, Time: 0.42s
Depth: 2 | Pruning: Enabled -> User Wins:   0, CPU Wins: 20, Ties:  0, Time: 0.63s
Depth: 3 | Pruning: Enabled -> User Wins:  14, CPU Wins:  6, Ties:  0, Time: 4.65s
Depth: 4 | Pruning: Enabled -> User Wins:   5, CPU Wins: 15, Ties:  0, Time: 3.76s
Depth: 5 | Pruning: Enabled -> User Wins:   4, CPU Wins: 16, Ties:  0, Time: 57.69s
Depth: 6 | Pruning: Enabled -> User Wins:   6, CPU Wins: 14, Ties:  0, Time: 23.79s
Depth: 1 | Pruning: Disabled -> User Wins:  16, CPU Wins:  4, Ties:  0, Time: 0.40s
Depth: 2 | Pruning: Disabled -> User Wins:   0, CPU Wins: 20, Ties:  0, Time: 1.06s
Depth: 3 | Pruning: Disabled -> User Wins:  13, CPU Wins:  7, Ties:  0, Time: 9.85s
Depth: 4 | Pruning: Disabled -> User Wins:   2, CPU Wins: 18, Ties:  0, Time: 47.90s
Depth: 5 | Pruning: Disabled -> User Wins:   9, CPU Wins: 11, Ties:  0, Time: 406.41s
```

Questions

1. **Increasing the depth of the algorithm allows it to evaluate moves further into the future, which can improve decision-making and therefore the probability of winning. However, deeper searches require significantly more time and increase the number of nodes visited, as the search space grows exponentially with depth. This trade-off between depth, time, and winning probability is critical to consider in games with large branching factors, as deeper searches may provide better decisions at the expense of computational efficiency.**
2. **Yes, it is possible to order child nodes to maximize pruning, particularly in the minimax algorithm with alpha-beta pruning. By sorting or arranging moves such that the most promising ones are evaluated first (e.g., by some heuristic such as the likelihood of leading to a winning state), we can often reach alpha or beta cutoffs sooner, reducing the number of nodes evaluated. This strategy is known as *move ordering* and is a common technique to enhance pruning effectiveness in alpha-beta pruning.**
3. **The branching factor is the average number of child nodes (or possible moves) from each node (or game state). In games like Connect4, the branching factor may decrease as the board fills up since fewer moves become available. Early in the game, the branching factor is higher as there are more possible moves, but as the game progresses and more slots are filled, the branching factor decreases, which reduces the number of possible moves.**
4. **Pruning in algorithms like alpha-beta pruning speeds up the search because it eliminates branches that cannot affect the final decision, allowing the algorithm to skip evaluating entire sections of the tree. This does not impact the accuracy of the result because pruned branches are guaranteed to be irrelevant to the optimal decision, thereby maintaining the same outcome but with improved efficiency.**
5. **When the opponent plays randomly, minimax is not optimal because minimax assumes an optimal opponent who aims to minimize the player's score, which does not apply in this case. An alternative algorithm like *expectimax* may be better suited here. Expectimax accounts for random moves by the opponent by calculating the expected value of moves, rather than assuming minimization by the opponent. This approach provides a more accurate evaluation when the opponent's actions are random, making it more appropriate than minimax for such situations.**